Classification & Model Evaluation

DAEN 300 - Fall 2026

Learning objectives

- Explain classification as the prediction of a category.
- Build a reproducible `scikit-learn` workflow.
- Evaluate predictions on unseen data.
- Interpret accuracy, confusion matrix, precision, recall, and F1.



References

- Textbook: *Python Data Science Handbook*, Ch. 5: Machine Learning, including Hyperparameters and Model Validation. `https://jakevdp.github.io/PythonDataScienceHandbook/05.00-machine-learning.html`
- Model evaluation (Scikit-learn User Guide): `https://scikit-learn.org/stable/modules/model_evaluation.html`
- Iris dataset (Scikit-learn): `https://scikit-learn.org/stable/modules/generated/sklearn.datasets.load_iris.html`

What is classification?

- **Supervised learning**
  inputs (features X) and known labels (target y).
- **Classification**
  predicts a category, not a continuous value.
- **Examples**
  email type, equipment fault category, flower species.

Dataset: Iris

- 150 observations, 4 measured features, 3 flower species.
- X: sepal length/width and petal length/width.
- y: species label.
- Built into `scikit-learn` (no external download required).

Training workflow

1. Load the dataset and inspect its shape and classes.
2. Split into training and test sets (e.g., 80% / 20%); stratify labels.
3. Fit the classifier **only on training data**.
4. Predict and evaluate on held-out test data.

A simple classifier: k-nearest neighbors

- k-NN predicts a class based on nearby training examples.
- Hyperparameter k: number of neighbors used for prediction.
- Start with $k = 5$
  (compare with different values, such as $k = 1$).
- High *training* accuracy does not guarantee good performance on new data.

Why separate training and testing?

- Evaluating on data used for fitting can be misleading.
- A held-out test set estimates performance on unseen examples.
- Keep test observations out of training and data-driven preprocessing.
- **Data leakage** can make test results seem better than they are.

Accuracy and the confusion matrix

$$\text{Accuracy} = \frac{\text{Number of correct predictions}}{\text{Total predictions}}$$

- Confusion matrix:
  rows = true classes, **columns = predicted classes**
  (scikit-learn convention).
- Diagonal entries: correct predictions.
- Off-diagonal entries: classes the model confuses.

Precision, recall, and F1

$$\text{Precision} = \frac{TP}{TP + FP}, \qquad \text{Recall} = \frac{TP}{TP + FN}$$

$$F_1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

- Precision: of predicted positives, how many are correct?
- Recall: of actual positives, how many are found?
- For multiple classes, inspect per-class results and macro averages.

PS. F1 refers to the F-score with $\beta = 1$, meaning precision and recall are given equal importance.

Validation and model improvement

- A single train/test split is a starting point, not a guarantee.
- Cross-validation evaluates performance on several subsets.
- Tune hyperparameters with training data / cross-validation, **not** the final test set.
  - Tuning hyperparameters means finding the best settings for a machine learning model to improve its performance.
    For instance:
  - K-Nearest Neighbors $\rightarrow$ Number of neighbors used for prediction.
  - Decision Tree $\rightarrow$ Maximum depth of the tree.
  - Random Forest $\rightarrow$ Number of decision trees.
  - Neural Network $\rightarrow$ How quickly the model updates its weights.
- Consider:
  - class imbalance (when one class has significantly more observations than another),
  - error costs (different types of prediction mistakes (false positive vs false negative),
  - and data leakage (information that would not legitimately be available at prediction time is used during model training or selection).

In-class lab: deliverable

- Open the provided Jupyter notebook;
  complete its TODO cells.
- Train one k-NN classifier on the Iris dataset.
- Report accuracy, confusion matrix, and classification report.
- Write brief explanations of your results **in your own words**.
- Submit the executed `.ipynb` **before class ends**.

Grading and AI

Holistic score (completeness and clarity):

- **4:** complete, correct, clearly explained.
- **3:** mostly complete; minor issues.
- **2:** partly complete; significant gaps.
- **1:** minimal or unclear work.

- AI may help with syntax, debugging, and concept explanations.
- You must run, verify, and **understand** your own notebook.
- A wholly AI-produced submission without student understanding is **NOT OK**.
- **Assignment work must be done in class.**